

How the application turns data into answers

A beginner-friendly technical guide to AccidentInfoAPI

Prepared for Md AI Walid | Code inspected on 3 October 2026

This guide follows the application in three stages: first the ETL pipeline prepares the data; next the frontend and REST API answer a user request; finally the database model explains how the records are stored and connected.

Each component is described using its actual filename and responsibility. The guide includes workflow diagrams, source-to-field mappings, example requests and an explanation of every required analytical question.

Part	What you will learn	Pages
1 ETL pipeline	Download, parse, transform, load, aggregate and record provenance.	2-7
2 User to answer	Frontend components, HTTP requests, middleware, routes, SQL and responses.	8-12
3 Database	Six tables, primary and foreign keys, normalized values and relationships.	13-16

Read this distinction first

The database prepares and supplies the answers; React displays them. Question titles, default selections and dataset identifiers are configuration. The result counts and rates are calculated from imported records.

The current application has no registration, login or administrator role checks. Operator tasks are described as maintenance work, not as protected admin-only features.

This guide documents the local code. It does not report newly measured accident totals, certify the live deployment, or claim that every source year is complete. Implementation limits are marked where they affect the explanation.

1 The complete ETL workflow

ETL means Extract, Transform and Load. Your implementation also separates parsing and aggregation. Extraction acquires files; parsing reads their structure; transformation creates consistent records; loading writes the database; aggregation calculates summaries.

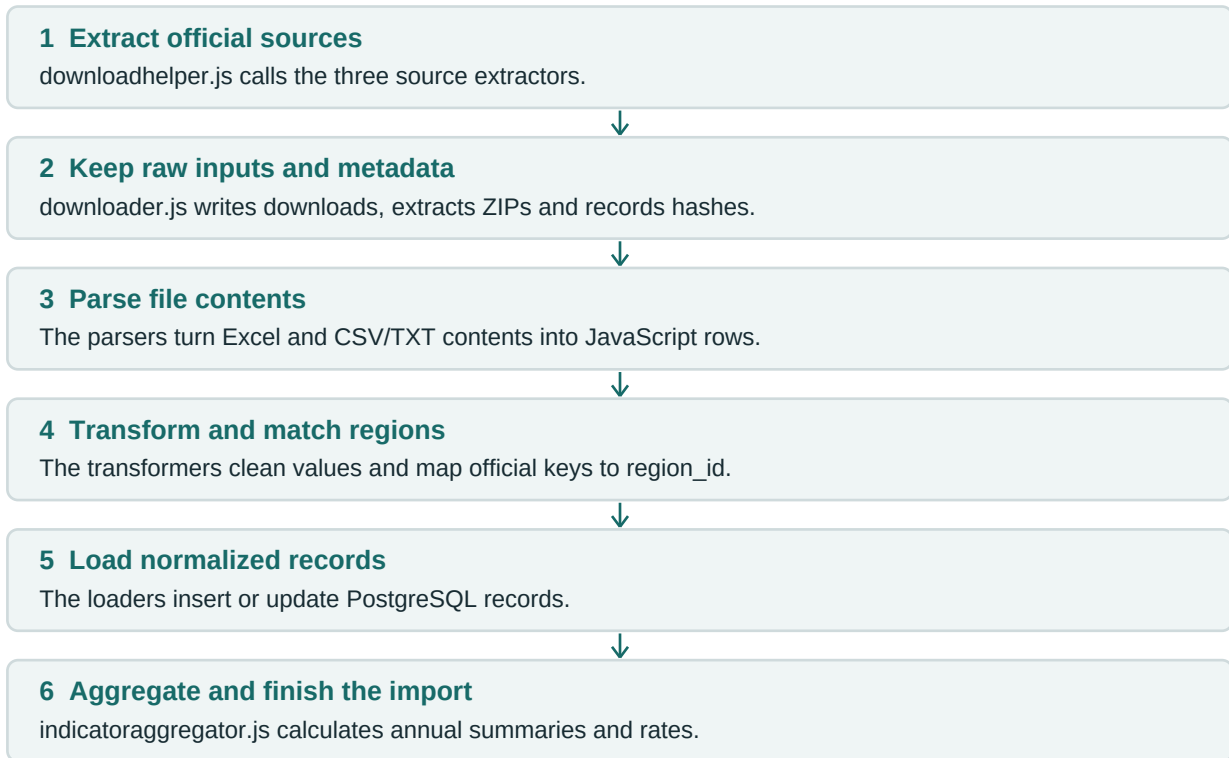


Figure 1. Extraction and the import command are separate steps.

The coordinator is runetl.js

backend/src/etl/runetl.js controls the import order. It creates an import_runs row, loads GV-ISys regions, assigns parents and creates an AGS-to-region-ID map. It then imports Regionalatlas values, loads accident files, seeds indicator definitions, builds aggregates and marks the run successful or failed.

Regions must come first because both accidents and indicator values need region_id. A source-file record is created before loading the corresponding source. The AGS map avoids looking up each region with a separate SQL query.

Code reference: backend/src/etl/runetl.js. Running ETL does not automatically run the downloader or initialize the schema.

2 Extractors and the download helper

The extraction files are in `backend/src/etl/extractors`. Their job is to obtain source material and evidence of its origin. They do not calculate the analytical answers.

Component	Responsibility
<code>downloadhelper.js</code>	<code>downloadAllSources</code> creates one Downloader and runs Unfallatlas, GV-ISys and Regionalatlas sequentially. It collects source outcomes and saves <code>download_manifest.json</code> .
<code>downloader.js</code>	Shared file operations: create folders, stream a download to a <code>.part</code> file, check size, retry, rename a completed download, compute SHA-256, extract ZIPs and write metadata JSON.
<code>baseextractor.js</code>	Shared base class for source logging and source-reference metadata. Each source class inherits these common operations.
<code>core/unfallatlas.js</code>	Discovers yearly CSV ZIP files from the NRW JSON index, with HTML discovery as a fallback. Selects years from 2016 onward, downloads and extracts archives, and saves reference material.
<code>core/gvisys.js</code>	Searches the Destatis page for dated Auszug_GV Excel links and chooses the newest match. If discovery fails, it uses the configured 31 December 2024 fallback.
<code>core/regionalatlas.js</code>	Downloads eight configured GENESIS statistical tables. It checks for HTML disguised as CSV and tries an alternate URL when applicable.

Which source contributes what

Unfallatlas supplies individual accident events. GV-ISys supplies region names, keys and hierarchy. Regionalatlas / Regionalstatistik supplies regional observations such as population, area, vehicle stock and passenger-car stock. Multiple sources are necessary because an accident record does not contain the number of cars registered in its region.

Reuse versus refresh

Without force, an existing file is reused and hashed. This avoids downloading it again, but does not check whether the provider changed the remote file. With `force=true`, the helper attempts a fresh download. The manifest contains nested failures, so a returned manifest alone is not evidence that every file succeeded.

Actual inputs are Excel and accident/statistical CSV or TXT, with ZIP and JSON/HTML discovery support. The ETL does not implement general Shapefile, XML or WMS ingestion.

3 Parsers read the source formats

A parser understands file structure. It does not save rows to PostgreSQL. The parsers return JavaScript objects that the transformers can work with.

Parser	Input and operation	Output
gvisysparser.js	Reads .xlsx using xlsx. Skips information sheets and reads the workbook data columns by position.	Region-source rows with satzart, land, rb, kreis, gem, name and statistical fields.
unfallatlasparser.js	Finds Unfallorte CSV/TXT files recursively. Uses csv-parser with a semicolon separator.	Accident-source rows such as UJAHR, UKATEGORIE and IstRad.
regionalatlasparser.js	Reads text as UTF-8 and splits semicolon fields. Recognizes population headers and supported date/year layouts.	Rows using AGS, Jahr, Wert and Merkmal.

Why GV-ISys rows can contain null

A workbook contains different kinds of records. A state record identifies a federal state; it is not the same as a municipality record. Fields such as municipality coordinates can therefore be absent. Seeing null on the first state row does not, by itself, mean the workbook failed to load.

The transformer deliberately keeps record types 10, 40 and 60 for states, districts and municipalities. Other record types are skipped because the chosen model does not represent every intermediate administrative level. Parsed row count and stored region count need not be equal.

Checks that matter during parsing

The ETL stops when no GV-ISys file is found or when region transformation yields zero records. Missing Regionalatlas files are logged and skipped. Accident files are read into arrays before transformation and loading; the CSV stream therefore does not make the complete pipeline memory-streaming.

The current accident filename year detector recognizes 2016 through 2024. Automatic download discovery can find later years, but that does not mean the parser accepts them. Regionalatlas parsing also depends on the expected layout and encoding; its simple field splitting is not a complete quoted-CSV implementation.

Code reference: backend/src/etl/parsers/*.js. Parser output is an intermediate representation, not the final database schema.

4 Transformers create common values

Transformation converts source-specific names and representations into one internal shape. It also matches administrative keys. A transformer returns objects; a loader later translates their camelCase names into database snake_case columns.

Transformer	Main work
regiontransformer.js	Cleans names, retains record types 10/40/60, builds state/district/municipality AGS keys, deduplicates by AGS and creates optional point geometry.
accidenttransformer.js	Converts numeric fields and flags, determines injury/fatal status, matches the best available region and attaches source-file and import-run IDs.
regionalatlastransformer.js	Finds region key, year and numeric value; checks configured total-row rules; resolves the AGS in the region map and emits an annual indicator value.

Source field	Transformed name / stored column	Rule
UJAHR	year / year	Numeric source year, or context year fallback.
UMONAT, USTUNDE, UWOCHENTAG	month, hour, weekday	Convert numeric text; missing values can be null.
UKATEGORIE	category; isPersonalInjury / is_personal_injury; isFatal / is_fatal	Keep code; categories 1/2/3 mean injury in the mapping; 1 sets fatal.
IstFuss, IstRad	isPedestrian / is_pedestrian; isBicycle / is_bicycle	1 or true becomes a boolean true.
IstPKW, IstKrad	isCar / is_car; isMotorcycle / is_motorcycle	Same boolean conversion.
XGCSWGS84, YGCSWGS84	longitude, latitude	Convert decimal text; use existing WGS84 fields.
AGS, Jahr, Wert	regionId, year, value	Resolve AGS to internal ID; set month = 0.

Region matching tries the eight-digit municipality AGS first, then the five-digit district code, then the two-digit state code. If none exists in the map, regionId is null. Broader matches retain the event but cannot support a precise municipality answer.

Current limits: no coordinate reprojection; category/type codes remain codes. Regionalatlas numeric conversion is applied twice and can remove a decimal point from an already numeric value. Decimal indicators require validation before use.

5 Loaders save data and aggregates

There are six loader files because storage involves more than the three source formats. Regions, event rows, indicator definitions, indicator values and provenance have different responsibilities. `bulk.js` is a shared helper, not another dataset.

Loader	Database work
<code>regionloader.js</code>	Upserts regions by ags in batches of 500. <code>updateRegionParents</code> links districts to states and municipalities to districts. <code>getRegionMap</code> returns AGS → <code>region_id</code> .
<code>accidentloader.js</code>	Upserts accidents in batches of 500 using <code>source_accident_key</code> . Stores time, codes, flags, coordinates, region and provenance IDs.
<code>regionalatlasloader.js</code>	Upserts an indicator definition and its values. Refreshes <code>regions.population</code> using the latest available population observations.
<code>indicatorloader.js</code>	Seeds definitions such as <code>total_accidents</code> , <code>fatal_accidents</code> and <code>bicycle_accidents</code> . It does not insert hardcoded answer totals.
<code>provenanceloader.js</code>	Creates and finishes <code>import_runs</code> records; hashes and records files in <code>source_files</code> .
<code>bulk.js</code>	Splits arrays and builds SQL placeholders plus parameter arrays for batch inserts. It does not access PostgreSQL itself.

What an upsert means

An INSERT with ON CONFLICT updates an existing row when its unique key already exists. For example, a region with an existing AGS is updated rather than inserted as a second region. Parameter arrays keep data values separate from SQL statement text.

What aggregation does

`aggregators/indicatoraggregator.js` groups accidents by attached `region_id` and year. It writes annual total, injury, fatal, pedestrian and bicycle counts to `indicator_values`. Annual rows use `month = 0`. It also calculates population and passenger-car rates when a denominator is available.

Aggregation reduces event rows to summary values; it does not replace the original events. Monthly values are supported by the schema, but this aggregator currently produces annual values. Ordinary count API routes still query accidents directly, and the passenger-car endpoint computes its own join and rate.

Population rates use `regions.population`, which is not tied to the accident year. General annual totals are not automatically rolled up to every ancestor level.

6 Running and reproducing an import

From the backend directory, configure DATABASE_URL for your PostgreSQL database, then run these commands in order. Schema initialization, downloading and ETL are separate operations.

```
npm install
npm run init-db
npm run download
npm run etl
npm run dev
```

init-db executes pgsq/schemas.sql. download writes source files and metadata. etl loads and summarizes the files already present. dev starts the server; it does not automatically run ETL. download:force requests new copies of existing downloads.

Evidence	Where to inspect it	Meaning
Download record	data/metadata and data/manifest	Requested URLs, timestamps, checksums and per-source outcomes.
Import history	import_runs	Start, finish, status and notes for each ETL execution.
File trace	source_files	Path, filename, source system, file hash and import association.
Loaded records	accidents and indicator_values	Foreign keys connect records to import history; some file links are incomplete.

What reproducibility currently guarantees

The code and source metadata make the processing steps inspectable. Upserts help repeat imports, but accident identity is a hash of absolute file path plus row number. Moving files can create different keys, and reordering or shortening a file can leave incorrect or stale associations. This is not stable official event identification.

A complete run is not wrapped in one transaction. If a later step fails, earlier batches can remain committed. Missing optional statistical files can be skipped and the run can still finish. Review each source outcome, imported years and unmapped regions rather than relying only on the word success.

Provenance that is not yet fully connected

The source-file loader supports source_url, but current runetl calls do not pass it. Dataset code, name and version fields are also not populated there. Regionalatlas values get import_run_id but not source_file_id. Download metadata therefore remains necessary for the full URL and retrieval history; imported_at is an import time, not the retrieval time.

The database connection currently requests SSL with certificate verification disabled. Local PostgreSQL without SSL is not compatible with this configuration unchanged. This guide does not alter the application or run an import.

7 What the frontend components do

Visitors go directly to the query page. There is no registration, login, token or session flow in the current application. React communicates with Express through HTTP; it does not connect directly to PostgreSQL.

Component	Responsibility
main.jsx and App.jsx	Mount React, import Bootstrap and local styles, track tabs and load the question catalog, options, states and schema map with Promise.all.
AppHeader.jsx	Switch Query, Schema and About views. Open the backend Swagger UI link.
QuestionConsole.jsx	Track the selected question, form values, result, error and loading state. Build query parameters and submit the selected endpoint.
QuestionForm.jsx	Render fields from the catalog. Boolean-type fields appear as Any / Yes dropdowns. Call supplied change and submit callbacks.
SelectField.jsx	Render database-derived years/states and other configured selectors. Translate category/type codes into readable labels when used.
ResponseFrame.jsx	Display data.answer as a count; arrays as tables; other objects as JSON. Offer the raw response for inspection.
SchemaExplorer.jsx	Display table purposes and keys from /schema-map.
AboutPage.jsx and DataLicensePanel.jsx	Explain the project and name the data providers. The notice refers readers to provider licence terms.
LoadingMessage.jsx	Show startup loading/error state. DocsPanel.jsx exists but is not mounted by App.
frontend/src/api.js	Select API_BASE, run fetch, parse JSON or text, and throw on unsuccessful HTTP status.

App stores shared metadata; QuestionConsole stores one question run. Changing state makes React render the updated form or answer. Changing a tab uses component state rather than navigating to a separate server-rendered page.

API_BASE uses VITE_API_BASE_URL or the configured Render fallback. Vite embeds the configured value at build time, so changing a deployment variable requires a new frontend build.

8 Follow one request end to end

Example: choose Simple custom accident count, select 2024, type Dresden in Region name and choose Bicycle = Yes. The year and name are filters; the application does not contain a prewritten Dresden answer.

```
GET /accidentinfoapi/answers/count?year=2024
&regionName=Dresden&bicycle=true
```

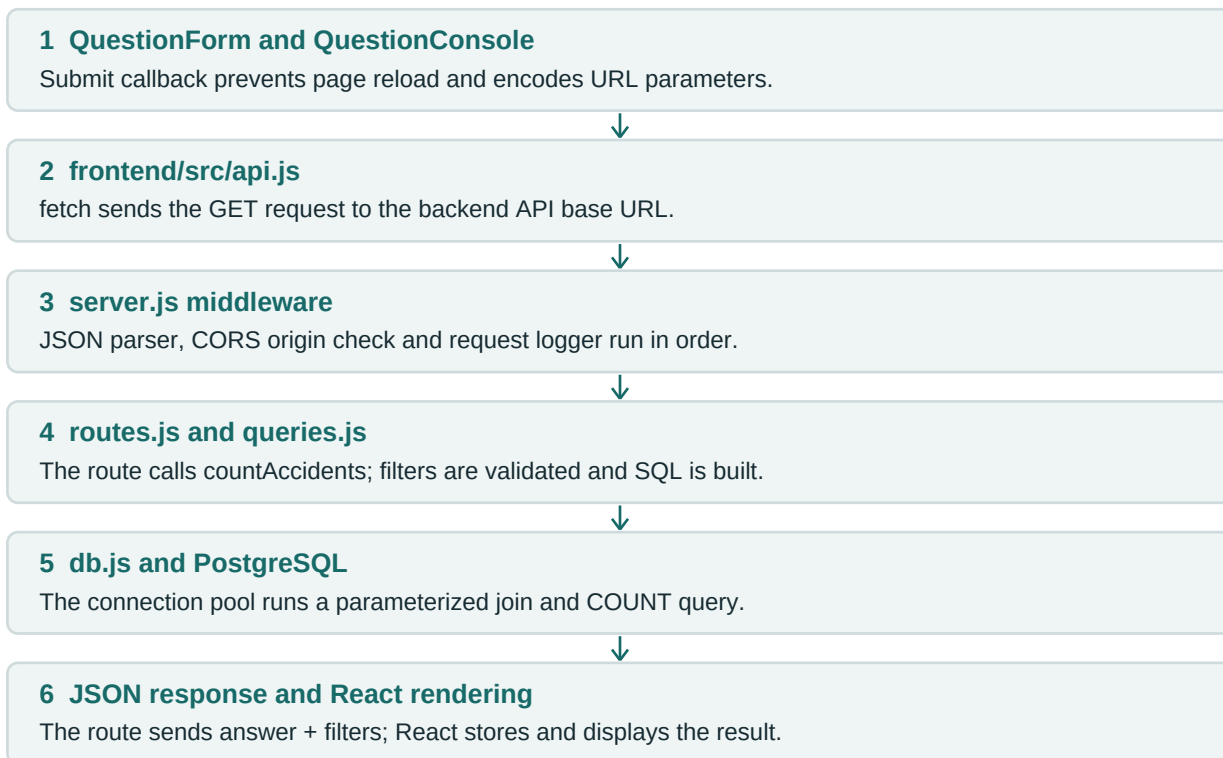


Figure 2. A browser question travels through the REST API to PostgreSQL.

The SQL joins accidents to its attached region, and optionally that region to its parent. It filters year = 2024, is_bicycle = TRUE, and a partial case-insensitive Dresden match on the region or parent name. Multiple filters are combined with AND.

routes.js wraps the result in an object with api, question and data. api.js reads the HTTP response. QuestionConsole stores it and ResponseFrame displays the number. If the request fails, asyncRoute forwards the error to Express and the frontend shows the returned error message.

9 API components and middleware

src is the source-code folder, not an endpoint name. Express exposes only registered routes. server.js mounts src/api/routes.js at /accidentinfoapi; the ETL and database folders remain internal modules.

File	Role in answering a request
server.js	Creates Express, installs middleware, mounts the API router and Swagger, exposes maintenance routes and starts listening.
src/api/routes.js	Matches the URL, reads req.query or req.params, calls a query function and prepares JSON. Its callbacks perform the controller role.
src/api/queries.js	Contains validation, analytical rules and SQL. It combines business logic and database queries; there is no separate service or controller directory.
src/api/catalog.js	Defines question titles, endpoint paths, fields, defaults and fixed filters. It controls the form, not the numerical results.
src/api/openapi.js	Builds the OpenAPI object displayed by swagger-ui-express. It documents routes but does not execute SQL or enforce validation.
src/db/db.js	Loads dotenv, creates a pg Pool using DATABASE_URL and exposes query, close and testConnection.

Middleware in execution order

express.json handles JSON bodies, although the analytical GET requests use URL parameters. cors checks permitted browser origins. The next middleware logs the time, HTTP method and URL. After route matching, the fallback handler returns 404 for an unknown route; the final error handler logs unexpected failures and returns JSON.

Validation and actual HTTP behavior

validateYear accepts integer years from 2000 to 2100. validateInteger checks month and limit ranges. Missing stateAgs is explicitly rejected with 400 on relevant routes. A missing region lookup returns 404. However, numeric validators throw errors marked 400 while the final handler always sends 500, so those validation errors currently surface as server errors.

CORS is not authorization. The server has no role checks, and GET /download and GET /etl remain callable despite the removal of frontend maintenance buttons. /etl returns 409 when its in-process running flag is already set. That flag does not lock other processes or CLI imports.

10 Where each question is answered

All paths below start with `/accidentinfoapi` and use GET. Change filter values to ask the same question for another imported year or region. These are example inputs, not hardcoded results.

Question	Path and example filters	Query function
Earliest accident year	<code>/answers/earliest-accident-year</code>	<code>earliestAccidentYear</code>
Personal injury in Saxony in 2023	<code>/answers/count</code> <code>year=2023, stateAgs=14, personalInjury=true</code>	<code>countAccidents</code>
First year for North Rhine-Westphalia	<code>/answers/available-from</code> <code>stateAgs=05</code>	<code>availableFrom</code>
First year for Mecklenburg-Western Pomerania	<code>/answers/available-from</code> <code>stateAgs=13</code>	<code>availableFrom</code>
Pedestrian accidents in Berlin in 2023	<code>/answers/count</code> <code>year=2023, stateAgs=11, pedestrian=true</code>	<code>countAccidents</code>
Accidents per 100000 passenger cars	<code>/answers/passenger-car-rate</code> <code>year=2023, limit=10</code>	<code>passengerCarRate</code>
Top five fatal districts in 2024	<code>/answers/top-fatal-districts</code> <code>year=2024, limit=5</code>	<code>topFatalDistricts</code>
Bicycle accidents in Dresden in 2024	<code>/answers/count</code> <code>year=2024, regionName=Dresden, bicycle=true</code>	<code>countAccidents</code>
Zero-event municipalities in Saxony in 2023	<code>/answers/zero-accident-municipalities</code> <code>year=2023, stateAgs=14</code>	<code>zeroAccidentMunicipalities</code>

Metadata and lookup endpoints

`/question-catalog` returns form definitions. `/metadata/options` returns available years, categories, types, states and regions. `/metadata/coverage` summarizes stored years, counts and source-file groups. `/regions` supports level, search and stateAgs and returns at most 500 records; `/regions/:ags` returns one region. `/schema-map` returns table/key descriptions.

`/health` reports API availability without testing a database query. `/openapi.json` returns the specification. At the server root, `/api-docs/` renders Swagger UI and `/status` reports in-memory maintenance state. Swagger is documentation, not an additional controller or database layer.

11 Read and justify the answers

Counts and first available years

The earliest-year function applies MIN(year) to accidents. State availability joins accidents.region_id to regions.region_id and filters the first two AGS characters before taking MIN. These answers describe the imported data, not necessarily the complete historical coverage of Germany.

An injury, pedestrian, bicycle or fatal answer counts accident events satisfying flags. It does not count injured people, pedestrians involved or people killed. Two participant flags can be true on one event; selecting both uses AND. Any leaves a flag unrestricted, while Yes requires true. There is no frontend No exclusion option.

Cross-source passenger-car rate

```
rate = accident_count / passenger_cars * 100000
```

The API groups accidents to district level where a municipality parent is available. It joins indicators using code passenger_cars_districts, then finds positive annual indicator_values for the same internal region ID. It chooses the year closest to the accident year; a tie selects the later year.

The response contains district_name, accident_year, passenger_car_year, accident_count, passenger_cars and accidents_per_100k_passenger_cars. Always report both years when they differ. This rate uses all counted events, not only events involving a passenger car. Missing denominators omit regions from the result.

Rankings and zero cases

Fatal ranking groups fatal event counts by district or fallback attached region and orders descending. Check that fallback regions are actually districts before presenting the list as a district-only comparison. Equal counts have no additional tie-break ordering in the current query.

Zero cases start with the full municipality hierarchy, LEFT JOIN matching accidents for the chosen year and keep groups with COUNT(accident_id) = 0. Incomplete coverage, missing hierarchy or events mapped only to a broader region can produce misleading zero cases.

Response shapes

Counts use data.answer and data.filters; earliest-year responses use data.earliest_accident_year; availability uses data.available_from_year. Rankings, rates and zero cases return data arrays. pg commonly returns SQL BIGINT/NUMERIC values as strings; countAccidents explicitly converts its answer to a JavaScript number.

To justify a result, retain the request filters, the SQL function, relevant table keys, source checksum/reference year and import run. This guide does not substitute an invented number for a live query.

12 The six tables and their roles

A canonical model is a common structure shared by different sources. Instead of copying each input file into a separate database table, the project separates events, administrative regions, measurements and provenance.

Table	One row represents	Main source
regions	One state, district or municipality identified by AGS.	GV-ISys; population can be refreshed from Regionalatlas.
accidents	One imported accident event.	Unfallatlas.
indicators	One measurement definition, with a code and unit.	Regionalatlas configuration or ETL calculation definitions.
indicator_values	One measurement for a region, indicator, year and month.	Regionalatlas observations or derived accident statistics.
import_runs	One ETL execution and its status.	The ETL coordinator and provenance loader.
source_files	One file reference recorded for an import.	Provenance loader and local file hash.

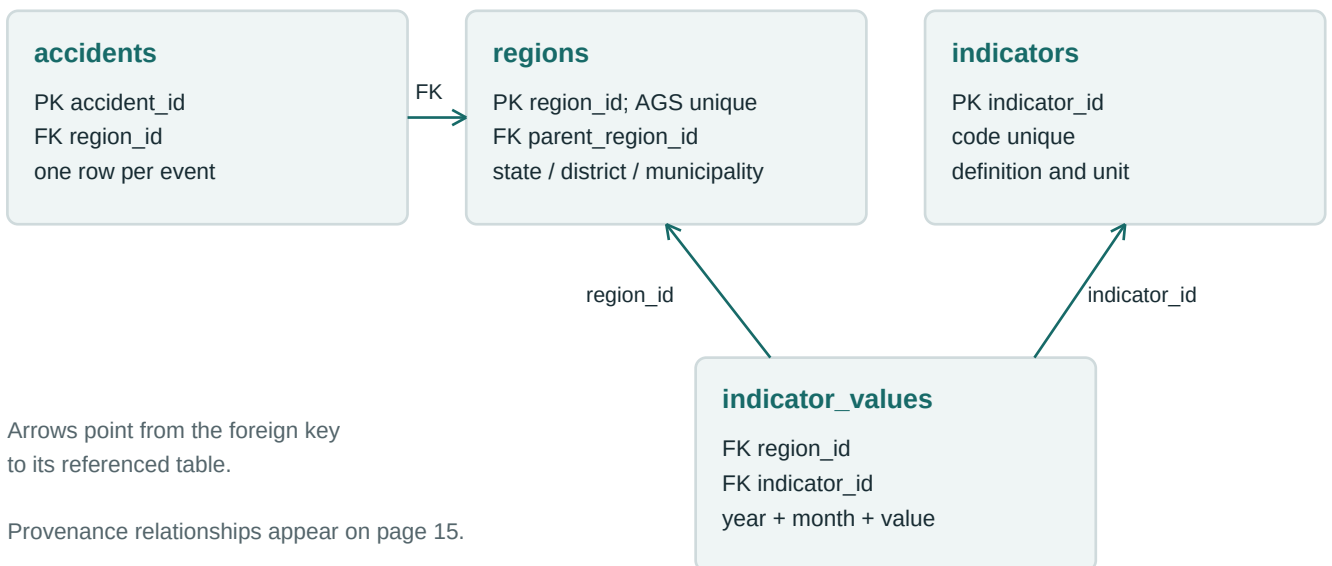


Figure 3. Core relationships. Many accident events can reference one region. Many values can reference the same region or indicator. Regions also have a self-referencing parent relationship.

PostgreSQL is suitable here because the questions depend on joins, groupings and consistent relationships. SQL calculates MIN, COUNT, GROUP BY and rates directly. Foreign keys protect links and unique constraints support upserts. The frontend never reads the tables directly.

13 Regions and accident records

regions

region_id is the internal SERIAL primary key. ags is a unique text key used to match official sources; keeping it as text preserves leading zeros. name stores the official label. level is checked against state, district, municipality or unknown. parent_region_id references another regions row.

The loader uses the first two AGS characters to connect a district to its state and the first five to connect a municipality to its district. source_file_id links the region to a source file. population is an optional integer convenience value, and created_at/updated_at record row timing.

geometry is TEXT, not a PostGIS geometry column. The transformer can write a GeoJSON-like point string when coordinates exist; it does not load district or state boundary polygons. The current API does not implement spatial intersection queries.

accidents

Column group	Stored fields and types
Identity	accident_id BIGSERIAL primary key; source_accident_key TEXT unique and required.
Time	year required INTEGER; month, hour and weekday optional INTEGER fields.
Source classifications	category, type and light_conditions are VARCHAR values. Numeric-looking source codes can be stored as text.
Participant and injury flags	is_personal_injury, is_fatal, is_pedestrian, is_bicycle, is_car, is_motorcycle are required BOOLEAN fields.
Location	longitude and latitude are NUMERIC; region_id is an optional foreign key to regions.
Origin and timing	source_file_id and import_run_id are foreign keys; created_at defaults to the database time.
Additional structure	participants is JSONB in the schema, but the current loader does not populate it.

The flags make common queries easy: bicycle questions filter is_bicycle, and fatal questions filter is_fatal. This is controlled duplication derived from source category/participant codes; consistency depends on the transformer and loader applying the same rules.

A null region_id is permitted by the schema. The API count query uses an INNER JOIN to regions, so unmatched events are excluded from that count even when no region filter is selected; earliestAccidentYear scans accidents directly.

14 Measurements and provenance

indicators and indicator_values

indicators separates the meaning of a measure from its observations. indicator_id is the primary key; code is unique; name, unit and source_system are required. description is optional. For example, passenger_cars_districts describes a measurement in cars, not a particular district count.

indicator_values stores the actual NUMERIC value. Each row has an indicator_value_id primary key, required region_id and indicator_id foreign keys, year and month. The unique combination (region_id, indicator_id, year, month) identifies an observation. month = 0 means annual; 1-12 means a month. The current aggregator writes annual observations.

import_runs and source_files

import_runs records import_run_id, started_at, finished_at, status and notes. The status is set to running at the start and success or failed at the end. The schema uses a text status field rather than a database-enforced enumeration.

source_files records source_file_id, import_run_id, source_system, source_url, file_path, file_name, dataset_code, dataset_name, dataset_year, dataset_version, sha256 and imported_at. The presence of a column does not guarantee it was filled: the current pipeline leaves several URL/version fields empty, as explained on page 7.

Referenced record	Referencing foreign key	Meaning
regions.region_id	regions.parent_region_id	Child administrative region to its parent.
regions.region_id	accidents.region_id; indicator_values.region_id	Events and measurements attached to a region.
indicators.indicator_id	indicator_values.indicator_id	Which measurement a numeric value describes.
import_runs.import_run_id	source_files, accidents and indicator_values: import_run_id	Which ETL run handled a file, event or observation.
source_files.source_file_id	regions, accidents and indicator_values: source_file_id	Which file originated a record, where populated.

Provenance keeps the origin inspectable; it does not automatically preserve a full history of previous event values. An upsert overwrites the current fields and may replace the import association. Raw file hashes and archived inputs are therefore valuable for reproducing a historical answer.

15 Keys behind every answer

Answer family	Tables and join keys	Calculation
Earliest year	accidents only	MIN(year).
State availability	accidents.region_id = regions.region_id	Match the AGS state prefix; take MIN(year).
Injury / pedestrian / bicycle count	accidents → regions on region_id; optional parent self-join	COUNT events with year, location and flag conditions.
Fatal districts	accidents → attached region → parent region	Group fatal event counts by the selected district/fallback ID.
Passenger-car rate	accidents → regions; indicator_values joins on region_id and indicator_id; indicators identifies the code	Divide grouped event count by positive car stock and multiply by 100000.
Zero municipalities	regions self-joins through municipality, district and state; LEFT JOIN accidents on region_id and year	Keep groups with zero matched accident IDs.
Source explanation	Events/values → import_runs and, where populated, source_files	Trace import execution and file evidence.

Constraints, indexes and the database layer

Primary keys identify rows. Unique keys prevent duplicate AGS values and duplicate indicator observations. Foreign keys enforce that a referenced record exists. Deleting a region or indicator cascades to its indicator_values; many event/provenance references instead use ON DELETE SET NULL. These rules matter before any data cleanup.

Indexes exist for accident year, month, region, combined year/region, flags and provenance references, plus region and indicator lookups. They support common filters but do not guarantee every query is fast; partial-name searches and large aggregations still need workload-specific evaluation.

db.js is the shared database abstraction: API queries and ETL loaders call db.query(sql, values), which delegates to the pg connection pool. init.js executes schemas.sql. CREATE TABLE IF NOT EXISTS initializes missing objects; it does not migrate existing tables to every later schema change.

Code reference

ETL: backend/src/etl/runetl.js; extractors/*; parsers/*; transformers/*; loaders/*; aggregators/indicatoraggregator.js.

API: backend/server.js; backend/src/api/routes.js, queries.js, catalog.js and openapi.js.

Database: backend/pgsql/schemas.sql; backend/src/db/db.js and init.js.

Frontend: frontend/src/App.jsx, api.js and components/* .jsx.

Configured official sources: opengeodata.nrw.de/produkte/transport_verkehr/unfallatlas/; unfallatlas.statistikportal.de; destatis.de (GV-ISys Gemeindeverzeichnis); regionalstatistik.de/genesis/online. Use the saved download metadata for exact file URLs and retrieval times.